

Recursion - 2

"The here and now is all we have, and if we play it right it's all we'll need."



~ Ann Richards



BRIGHT
DROPS
.com

Good
Evening ☺

Today's Content

- (a) Sum of digits
- (b) Power function
- (c) Fast power function } → V.V.V.I interview
- (d) TC / SC of recursive code → class 3

In your off day → Give you a hint on the communication channel for the least PSP question

Basics

Recursion → Function calling itself

↳ Solving problems with sub problem

↳ smaller instance of same problem

$$\text{sum}(5) = 1 + 2 + 3 + 4 + 5$$

$$\boxed{\text{sum}(4)} + 5$$

sub problem

$$\text{sum}(n) = 1 + 2 + 3 + 4 + \dots + n-1 + n$$

$$\boxed{\text{sum}(n-1)} + n$$

sub problem

Steps to Recursive Code

Assumption: Decide what your function suppose to do

Main logic : Solving assumption using subproblem

Base condition: Inputs for which we want to stop recursion

Digits sum

$\text{sumD}(n) \rightarrow$ sum of all digits of number n

$$\text{sumD}(789) \Rightarrow 7 + 8 + 9 = 24$$

$$\text{sumD}(235) \Rightarrow 2 + 3 + 5 = 10$$

$$\text{sumD}(12345) = 1 + 2 + 3 + 4 + 5 = 15$$

$$\text{sumD}(7) = 7 \Rightarrow 7$$

$N = d_1 d_2 d_3 d_4 \dots d_{a-1} d_a$ // Each d is representing a digit

$$\text{sumD}(N) = d_1 + d_2 + d_3 + d_4 + \dots + d_{a-1} + d_a$$

$$\text{sumD}(N) = \text{sumD}(a-1) + d_a$$



$$\text{sumD}(N) = \text{sumD}(N/10) + N\%10$$

→ Tracing {TODO}

```
int sumD(N)
```

```
    if (n == 0) return 0;
```

```
    return sumD(N/10) + N%10
```

```
}
```

OR

```
int sumD(N)
```

```
    if (n < 10) return n;
```

```
    return sumD(N/10) + N%10
```

```
}
```

Base condition

$\text{sumD}(7) = \text{sumD}(7/10) + 7\%10$

$\text{sumD}(0) = \text{sumD}(0) + 0\%10$

$n < 0$

$n = -123 \Rightarrow 6$

$n = 123 \Rightarrow 6$

Note :- if ($n < 0$) $\rightarrow$ make it positive before calling the sumD function.

Tracing

```
main {
```

```
    n = 563
```

```
    print (sumD(n));  $\Leftrightarrow$  print (14)
```

```
}
```

```

int sumD(n)    n=563
|
| if (n < 10) return n;
|
| return sumD(n/10) + n%10
|
|

```

11 + 3

```

int sumD(n)    n=56
|
| if (n < 10) return n;
|
| return sumD(n/10) + n%10
|
|

```

5 + 6

```

int sumD(n)    n=5
|
| if (n < 10) return n;
|
| return sumD(n/10) + n%10
|
|

```

Base condition

↳ look for the last
valid input

for which the
main logic fails

* Print A to 1

A = 5 → 5 4 3 2 1

PrintAto1(5) → 5 4 3 2 1
 ↳ SOP(5):
 PrintAto1(4)

print Atoi(n) $\rightarrow$ n n-1 n-2 ... 3 2 1

$\searrow$ sop(n);

printAtoi(n-1);

```
void solve (n) {
```

Print Atoi(n)

System.out.println();

2

```
void printAtoi(n)
```

```
if (n == 0) {
```

return ;

3

```
System.out.print(n);
```

```
print Atoi (n-1);
```

,

* pow Way 1 $\rightarrow$ Note :- Do not worry about overflows
 $\downarrow$
 a^n

$$\text{Pow}(a, n) = \underbrace{a * a * a * a * \dots * a}_{n \text{ times}} = a^n$$
$$\text{Pow}(a, 4) = a * a * a * a = a^4$$

$$\text{Pow}(a, 7) = \underbrace{a * a * a * a * a * a}_{\text{6 times}} * a = a^7$$

$$\text{Pow}(a, 7) = \text{Pow}(a, 6) * a$$

$$\text{Pow}(a, n) = \underbrace{a * a * a * a * a \dots * a}_{\text{n-1 times}} * a$$

$$\text{Pow}(a, n-1) * a$$

$a > 0$ ← $n \geq 0$
 long pow(int a, int n) { // Ass → Given a & n, calculate & return a^n
 | if(n == 0) return 1;
 | return pow(a, n-1) * a;
 3

Base condition

Note:- We can't use if(n==1) as base condition

Tracing → {TODO}

no need to worry about int & long range

way 2 : Note :- Do not worry about overflows

$$\text{Pow}(a, n) : \underbrace{a * a * a * a * \dots * a}_{n \text{ times}} = a^n$$

$$\text{Pow}(a, 6) : \underbrace{a * a * a}_{a^3} * \underbrace{a * a * a}_{a^3} = a^6 \Rightarrow \text{Weg 1} \\ \text{pow}(a, 5) * a$$

$$\text{Pow}(a, 6) : \text{pow}(a, 3) * \text{pow}(a, 3)$$

$$\text{Pow}(a, 10) : \text{pow}(a, 5) * \text{pow}(a, 5)$$

$$\text{pow}(a, 7) : \underbrace{a * a * a}_{\text{pow}(a, 3)} * \underbrace{a * a * a}_{\text{pow}(a, 3)} * a$$

$$\text{Pow}(a, 11) : \text{pow}(a, \frac{11}{2}) * \text{pow}(a, \frac{11}{2}) * a$$

$\text{Pow}(a, n) :$ if n is even
 $\quad \rightarrow \text{pow}(a, n/2) * \text{pow}(a, n/2)$
 if n is odd
 $\quad \rightarrow \text{pow}(a, n/2) * \text{pow}(a, n/2) * a$

```
long pow2 (int a, int n){
```

```
    if (n == 0) return 1L;
```

```
    if (n/2 == 0)
```

```
        return pow2(a, n/2) * pow2(a, n/2);
```

```
    else
```

```
        return pow2(a, n/2) * pow2(a, n/2) * a;
```

```
}
```

Note.:- $\text{Pow2}(a, n/2)$ is a subproblem, calculated multiple times.

→ Store the ans in a variable, & then use the variable

```
long pow3 (int a, int n){
```

```
    if (n == 0) return 1L;
```

```
    long t = pow2(a, n/2);
```

```
    if (n/2 == 0) return t * t;
```

```
    else return t * t * a;
```

```
}
```

Tracing

→ Ans = 128

```
long pow3(a, n)
```

```
if (n == 0) return 1;
```

```
t = pow3(a, n/2)
```

```
if (n % 2 == 0)
```

```
{ return t * t
```

```
}
```

```
else {
```

```
return t * t * a;
```

```
}
```

8 * 8 * 2

a = 2 n = 7

$$2^7 = 128$$

8

```
long pow3(a, n)
```

```
if (n == 0) return 1;
```

```
t = pow3(a, n/2) = 2
```

```
if (n % 2 == 0)
```

```
{ return t * t
```

```
}
```

```
else {
```

```
return t * t * a;
```

```
}
```

2 * 2 * 2

a = 2 n = 3

a = 2, n = 1

```
long pow3(a, n)
```

```
if (n == 0) return 1;
```

```
t = pow3(a, n/2) = 1
```

```
if (n % 2 == 0)
```

```
{ return t * t
```

```
}
```

```
else {
```

```
return t * t * a;
```

```
}
```

1 * 1 * 2

```
long pow3(a, n)
```

```
if (n == 0) return 1;
```

```
    t = pow3(a, n/2)
```

```
    if (n % 2 == 0)
```

```
    |    return t * t
```

```
    }
```

```
    else {
```

```
    |    return t * t * a;
```

```
    }
```

$a = 2$

$n = 0$

Pre-requisite

Any number $\% M = [0 \ m-1]$

$(x * y) \% m = (x \% m * y \% m) \% m$

* Power with modulus : Note $\rightarrow$ Take care of overflows

$\text{pow}(a, n, m) = a^n \% m$

Constraints

$\rightarrow 1 \leq a \leq 10^9$

$\rightarrow 1 \leq n \leq 10^9$

$\rightarrow 1 \leq m \leq 10^9$

if (n is even)

$\hookrightarrow a^n \% m = \{ a^{n/2} * a^{n/2} \} \% m$

$= \{ a^{n/2} \% m * a^{n/2} \% m \} \% m$

$\text{Pow}(a, n, m) = \{ \text{Pow}(a, n/2, m) * \text{Pow}(a, n/2, m) \} \% m$

if (n is odd)

$$\hookrightarrow a^n \% m = \{ a^{n/2} * a^{n/2} * a \} \% m$$

$$= \{ a^{n/2} \% m * a^{n/2} \% m * a \% m \} \% m$$

$$\text{Pow}(a, n, m) = \{ \text{pow}(a, n/2, m) * \text{pow}(a, n/2, m) * a \% m \} \% m$$

Find Exp

$\text{pow}(a, n, m) \rightarrow$

if (n % 2 == 0)

return $\{ \text{pow}(a, n/2, m) * \text{pow}(a, n/2, m) \} \% m$

}

else

return $\{ \text{pow}(a, n/2, m) * \text{pow}(a, n/2, m) * a \% m \} \% m$

}

Note. :- $\text{Pow2}(a, n/2, m)$ is a subproblem, calculated multiple times.

→ Store the ans in a variable, & then use the variable

Find Find Exp

pow(a, n, m)

```
    t = pow(a, n/2, m);  
    if (n % 2 == 0)  
    |   return (t * t) % m  
    |  
    }  
    else {  
    |   return (t * t * a % m) % m  
    |  
    }  
}
```

Constraints

$$\rightarrow 1 \leq a \leq 10^9$$

$$\rightarrow 1 \leq n \leq 10^9$$

$$\rightarrow 2 \leq m \leq 10^9 \longrightarrow \max m = 10^9$$

$$\text{any no. \% } m = 10^9 - 1$$

$$\approx 10^9 \text{ max}$$

$$\text{int} \rightarrow 10^9$$

$$\text{long} \rightarrow 10^{18}$$

$$m == 1 \rightarrow \text{Ans} = 0$$

long powmod (int a, int n, int m)

if (n == 0) return 1

long t = powmod(a, n/2, m) // $t = (a^{n/2} \% m)$
 $\hookrightarrow \text{max} = 10^9$

if (n % 2 == 0)

return (t * t) % m // $[10^9 * 10^9] \% m$

else

$= \frac{10^{18}}{\text{store in long}} \% m$

return (t * t * a % m) % m

$(10^9 * 10^9 * 10^9) \% m$

10^{27} can't be stored in

int or long $\hookrightarrow$

Overflow
error

$\Rightarrow (t * t * a \% m) \% m$

$(x * y) \% m$

Expansion

$(x \% m * y \% m) \% m$

$(a \% m) \% m = a \% m$

$$\Rightarrow ((t * t) \% m * (a \% m) \% m) \% m$$

$$\Rightarrow ((t * t) \% m * (a \% m)) \% m$$

$$\underbrace{10^{18} \% m}$$

$$(10^9 * 10^9) \% m$$

$$\underbrace{10^{18} \% m} \Rightarrow 10^9$$

if $(x < 0)$ $\Rightarrow$ To calculate in Java

$$\rightarrow (x \% m + m) \% m$$

Final Code

```
main ( )
```

```
    long ans = powmod(a, n, m)
```

```
    ans = (ans \% m + m) \% m
```

```
long powmod (int a, int n, int m)
```

```
    if (n == 0) return 1
```

```
    long t = powmod(a, n/2, m)
```

```
    if (n % 2 == 0)
```

```
        return (t * t) % m
```

```
    }
```

```
    else
```

```
        return ((t * t) % m * a % m) % m
```

```
    }
```

```
}
```

$$\underbrace{-2 \% 7} \Rightarrow \underline{\hspace{2cm}}$$

↪ [0 6]

$$\underbrace{-2^3 \% 7} \Rightarrow \underline{\hspace{2cm}}$$

↪ [0 6]

1 to A

```
void inc(n)
|
|   if (n == 1){
|       print(1);
|       return;
|   }
|   inc(n-1)
|   print(n)
|
| }
```